

Revenue Analysis

Database & Table Creation

Create database Sales_Analysis;

create or replace table products(ProductID integer primary key, ProductName VARCHAR(50), Category varchar(50));

create or replace table store(StoreID integer primary key, StoreName Varchar(100), Region varchar(100));

Create or replace table DateDim (Date Date PRIMARY KEY, Year INTEGER, Month INTEGER,MonthName VARCHAR(20),Quarter INTEGER, IsCurrentQuarter BOOLEAN);

create or replace table sales(TransactionID integer primary key, Date date, StoreID integer, ProductID integer, UnitsSold integer, UnitPrice decimal(10,2),
constraint fk_sales foreign key (ProductID) references products(ProductID),
constraint fk_store foreign key (StoreID) references store(StoreID),
constraint fk_date foreign key (Date) references Datedim(Date)
);

insert INTO products values
(5001, 'T-shirt', 'Apparel'),
(5002, 'Baseball Cap', 'Accessories'),
(5003, 'Hoodie', 'Apparel'),
(5004, 'Smartwatch', 'Electronics'),
(5005, 'Jeans', 'Apparel'),
(5006, 'Sneakers', 'Apparel'),
(5007, 'Belt', 'Accessories'),
(5008, 'Sunglasses', 'Accessories'),
(5009, 'Tablet', 'Electronics'),
(5010, 'Headphones', 'Electronics'),
(5011, ' T-shirt ', 'Apparel'),
(5012, 'hoodie', 'apparel'),
(5013, 'SMARTWATCH', 'ELECTRONICS'),
(5014, NULL, 'Accessories'),
(5015, 'Cap', NULL);

insert into Store values

(101, 'Downtown Mall', 'North'),
(102, 'Uptown Plaza', 'South'),
(103, 'City Center', 'East'),
(104, 'Lakeside Outlet', 'West'),
(105, 'Riverside Mall', 'North'),
(106, 'Central Square', 'South'),
(107, 'Hilltop Store', 'East'),
(108, 'Bayview Plaza', 'West'),
(109, 'Downtown Mall', 'north'),
(110, 'Airport Kiosk', NULL),
(111, NULL, 'West'),
(112, 'Uptown Plaza', 'South');

insert into sales values

(1, '2024-03-01', 101, 5001, 10, 20.00),
(2, '2024-03-02', 102, 5002, 5, 15.00),
(3, '2024-04-15', 101, 5003, 7, 35.00),
(4, '2024-05-10', 103, 5001, 12, 19.50),
(5, '2024-06-21', 104, 5004, 3, 120.00),
(6, '2024-07-05', 101, 5001, 8, 20.00),
(7, '2024-07-06', 102, 5002, 0, 15.00),
(8, '2024-07-07', 103, 5003, 6, 34.99),
(9, '2024-08-01', 104, 5004, 2, 119.99),
(10, '2024-08-15', 101, 5002, 4, 15.00),
(11, '2024-09-01', 102, 5001, 9, 20.00),
(12, '2024-09-12', 103, 5004, 1, 125.00),
(13, '2024-10-01', 104, 5003, 5, 35.00),
(14, '2024-10-05', 101, 5001, 10, 20.00),
(15, '2024-10-05', 101, 5001, 10, 20.00),
(16, '2024-11-11', 102, 5002, 3, NULL),
(17, '2024-11-12', 103, 5003, NULL, 35.00),
(18, '2024-12-01', 104, 5004, 50, 120.00),
(19, '2024-12-15', 101, 5002, -2, 15.00),
(20, '2024-12-20', 102, 5001, 6, 19.50);

```
insert into DateDim
select
    d AS Date,
    YEAR(d) AS Year,
    MONTH(d) AS Month,
    MONTHNAME(d) AS MonthName,
    QUARTER(d) AS Quarter,
    QUARTER(d) = QUARTER(CURRENT_DATE()) AS IsCurrentQuarter
FROM (
    SELECT DATEADD(DAY, SEQ4(), '2024-01-01') AS d
    FROM TABLE(GENERATOR(ROWCOUNT => 366)) -- 2024 is leap year
)
WHERE d <= '2024-12-31';
```

-----sql server version-----

```
with datecte as
(
    select cast('2024-01-01' as date) as d
    union all
    select dateadd(day, 1, d) from datecte
    where d < '2024-12-31'
)
```

```
insert into datedim
select
    d as [date],
    year(d) as [year],
    month(d) as [month],
    datename(month, d) as monthname,
    datepart(quarter, d) as [quarter],
    case
        when year(d) = year(getdate())
            and datepart(quarter, d) = datepart(quarter, getdate())
        then 1 else 0
    end as iscurrentquarter
from datecte
option (maxrecursion 366);
```

Data Cleaning

```
select * from sales;
```

```
select * from store;
```

```
select * from products;
```

```
select * from datedim;
```

--- Lets understand the dataset taking one table at a time.

```
select * from products;
```

```
select count(*) from products;
```

```
select count(productname) from products;
```

--- there is a difference between count(*) and count(columnname) that shows we 1 null in productname.

```
select count(productid) from products;
```

-- no nulls in primary key.

```
select count(distinct(Productname)) from products;
```

```
select distinct(productname) from products;
```

--- we can see 2 hoodie not matching the capitalize case letter. 2 smartwatch one upper one lowercase

--- two t-shirt with trailing spaces in them.

--- lets clean the productname column.

```
update products  
set productname = UPPER(trim(productname));
```

--- lets check whether the update is successfull or not.

```
select distinct(productname) from products;
```

-- yes the t-shirt with trailing spaces is gone. & smartwatch is upper letter as required

```
select count(category) from products;
```

```
select count(*) from products;
```

--- again the difference shows the presence of null which we will be taking care later

```
select distinct(category) from products;
```

--- some are lower case some are upper case some are capitalized which shows inconsistency lets do upper case in all.

```
update products  
set category = upper(category);
```

--- to confirm lets check.

```
select distinct(category) from products;
```

--- yes every category is now in upper case. with 3 distinct category and 1 null to be noted.

```
select * from products;
```

--- lets jump to nulls.

--- looking at the table the best usecase of nulls here would not be removing them but to replacing them with "unknown".

```
update products  
set productname = 'UNKNOWN' where productname is null;
```

```
update products  
set Category = 'UNKNOWN' where category is null;
```

---- final check

```
select count(*) from products where ProductID IS null  
OR ProductName IS null OR Category is null;
```

--- count(0) means no nulls

--- lets check for duplicates

```
select * from products;
```

```
select productid, count(*) from products group by productid having count(*) > 1;
```

-- no result that means the product id has no duplicates.

```
with cte as (  
select productname ,category , row_number() over( partition by productname ,category  
order by productname ,category) as rn from products )
```

```
delete from cte where rn> 1;
```

----or use

```
DELETE FROM products  
where productid IN (  
select productid  
From (  
SELECT productid, row_number() OVER (PARTITION BY productname, category ORDER BY  
productid ) AS rn FROM products)  
WHERE rn > 1  
);
```

--- lets check for validation.

```
with cte as (  
select productname ,category , row_number() over( partition by productname ,category  
order by productname ,category) as rn from products )
```

```
select * from cte where rn> 1;
```

```
select * from products;
```

----now we have our products table cleaned and ready.---

```
select * from store;
```

```
select count(*) from store;
```

```
select count(storeid) from store;
```

---- both the count gave the same number that means there are no nulls in the storeid ie primarykey

```
select count(storename) from store;
```

--- it outputed 11 that means there is 1 null so lets validate it

```
select count(*) from store where storename is null;
```

--- outputed exactly 1 row..

--- given thencontext we can either remve the null or replace it with unknown.

--- lets replace it with "Unknown" as deleting the null is not thye correct choice due to the less the data we have and deleting would even remove the rows that could come in handy later.

```
update store
```

```
set storename = 'UNKNOWN' where Storename is null; --- replacing nulls with "Unknown" as it is a categorical column
```

---lets validate it again by checvking for nulls

```
select * from store where storename is null;
```

--- query produced no result meaning the updation was successful.

```
select count(distinct(storename)) from store;
```

```
select count(storename) from store;
```

```
select distinct(storename) from store;
```

```
select storename from store;
```

--- inspecting the data we see duplicate in column but lets see are there any duplicate in whole!

```
select storename, region, count(*) from store group by storename, region having count(*)>1;
```

--- thisshows there is infact one row duplicated lets remove that row.

```
with cte as (select storename, region, count(*) from store group by storename, region having count(*)>1)
delete from cte
```

---or use

```
delete from store where storeid in
(select storeid from
(select storeid , row_number() over(partition by storename, region order by storename,
region) as rn from store)
where rn> 1)
```

--- lets validate

```
select storename , region , count(*) from store group by storename, region having count(*)
> 1;
```

--- duplicate deleted

```
update store
set storename = ltrim(storename); ---removing extra trailing spaces in the left of the word.
```

--- lets inspect region column

```
select count(region) from store; ---10 (expected)
```

```
select count(distinct(region)) from store; ---5 (okay lets investigate further)
```

```
select distinct(region) from store;
```

--- investigating the data a we find 1 null and 1 entry with is lower case but all are capitalise.
--- lets change that.

```
update store
set region = 'UNKNOWN' where region is null;
```

--- lets validate

```
select * from store where region is null;
```

```
update store
set region = UPPER(trim(region)); ---capitalise the first letter and removing extra trailing
spaces.
```

```
select distinct(region) from store;
```

--- Capitalisation done


```
select * from store;  
--- Store Table is ready to use.
```

--- lest check the sales table.

```
select * from sales;
```

--- lets check for nulls all column at a time using OR Operator.

```
select * from sales where transactionid is null or  
date is null or storeid is null or productid is null or  
unitssold is null or unitprice is null;
```

----as we have nulls in numerical column so now we have to decide wether to impute it with mean or remove the nulls.

```
update sales  
set unitssold = (select avg(unitssold) from sales) where unitssold is null; --- similary based on  
the condition we can use min, max, sum.
```

--- lets validate

```
select * from sales where unitssold is null;
```

--- null removed from Unitssold;

```
update sales  
set unitprice = (select avg(Unitprice) from sales ) where unitprice is null;
```

-- lets valiadtde

```
select * from sales where unitprice is null;  
---- nulls removed
```

```
select * from sales;
```

```
select count(*) from sales;
```

```
select count(distinct(*)) from sales;
```

---no difference no duplicates.

```
select distinct(unitssold) from sales;
```

--- negative value in unitssold is not possible.

--lets investigate further

```
select * from sales where unitssold < 0
```

---- unitssold = -2 could be a data entry error and we can assume its 2 so lets convert all the negative value to positive

update sales

set unitssold = abs(unitssold); ----abs converts all the negative values to positive values

```
select * from sales where unitssold < 0;
```

---- converted ---

---- lest investigate the same for unitprice

```
select * from sales where unitprice < 0;
```

--- no such records

```
select * from sales;
```

--- sales table is ready for analysis.

----- now as we understood the data lets answere the business questions.

```
select * from products;
```

```
select * from store;
```

```
select * from datedim;
```

Business Solution

-----total revenue per store in last quarter-----

select quarter(max(date)) from sales; --- quarter for max(date) but quarter doesnt
work in sql swerver so we can use --
datepart(quarter, max(date))

select productid, (unitsold*unitprice) as revenue, date from sales where quarter(date) in
(select quarter(max(date)) from sales)
group by productid, revenue, date;

select st.storename, sum(s.unitsold*s.unitprice) as revenue from sales
as s join store as st on s.storeid = st.storeid
where quarter(date) in (select quarter(max(date)) from sales)
and year(date) in (select year(max(date)) from sales)
group by st.storename;

----one thing to keep in mind that quarter(date) doesnt work in sqlserver so for that we
need to use Datepart(Quarter, date)

<u>STORENAME</u>	<u>REVENUE</u>
Uptown Plaza	247.11
City Center	280.00
Lakeside Outlet	6175.00
Downtown Mall	430.00

----lets start finding the top 5 products based on revenue-----

```
select p.productname, sum(s.unitssold * s.unitprice) as revenue from sales as s
left join products as p on s.productid = p.productid
group by p.productname
order by revenue desc
limit 5;
```

----- another solution -----

```
with outer_cte as(
with cte as (
select s.*, p.* from sales as s
left join products as p on s.productid = p.productid)
select productname, sum(unitssold * unitprice) as revenue ,
dense_rank() over( order by revenue desc) as rnk from cte
group by productname)
select * from outer_cte where rnk <= 5;
```

-----sqlserver solution----

```
select top 5 p.productname, sum(s.unitssold * s.unitprice) as revenue from sales as s
left join products as p on s.productid = p.productid
group by p.productname
order by revenue desc;
```

---using cte inside another cte is not allowed in sqlserver for that we have to create different cte for each step

PRODUCTNAME	REVENUE
-------------	---------

Smartwatch	6724.98
------------	---------

T-Shirt	1291.00
---------	---------

Hoodie	909.94
--------	--------

Baseball Cap	295.11
--------------	--------

----monthly sales trend-----

```
select year(s.date) as sales_year, month(s.date) as sales_month, sum(s.unitsold*  
s.unitprice) as revenue from sales as s  
left join products as p on s.productid= p.PRODUCTID  
where p.productname = 'Baseball Cap'  
group by year(s.date), month(s.date)  
order by sales_month asc, sales_year asc;
```

SALES_YEAR	SALES_MONTH	REVENUE
2024	3	75.00
2024	7	0.00
2024	8	60.00
2024	11	130.11
2024	12	30.00

-----best selling product by revenue category wise-----

```
select productname, category, revenue from (select p.productname,p.category,
sum(s.unitssold * s.unitprice) as revenue,
dense_rank() over(partition by p.category order by revenue) as rnk from sales as s
left join products as p on s.productid = p.productid
group by p.productname, p.category
order by revenue desc)
where rnk = 1;
```

PRODUCTNAME	CATEGORY	REVENUE
Smartwatch	ELECTRONICS	6724.98
Hoodie	APPAREL	909.94
Baseball Cap	ACCESSORIES	295.11

----parameterising category---

```
set category_name = 'APPAREL';
```

```
select p.productname, sum(s.unitssold * s.unitprice) as revenue from sales as s  
left join products as p on s.productid = p.productid  
where category = $category_name  
group by p.productname, p.category  
order by revenue desc;
```

----SQL SERVER SOLUTION

```
declare @category_name varchar(50) = 'APPAREL';
```

```
select p.productname, sum(s.unitssold * s.unitprice) as revenue from sales as s  
left join products as p on s.productid = p.productid  
where category = @category_name  
group by p.productname, p.category  
order by revenue desc;
```